

作业答案-继承和多态

一、简答题

1. 在 C++ 中，protected 类成员访问控制有什么作用？

答：C++ 中引进 protected 成员保护控制，缓解了数据封装与继承的矛盾。在基类中声明为 protected 的成员可以被派生类使用，但不能被基类的实例用户使用，这样能够对修改基类的内部实现所造成的影响范围（只影响子类）进行控制。protected 成员保护控制的引进使得类有两种接口：与实例用户的接口和与派生类用户的接口。

2. 在 C++ 中，三种继承方式各有什么作用？

答：类的继承方式决定了派生类的对象和派生类的派生类对基类成员的访问限制。public 继承方式使得基类的 public 成员可以被派生类的对象访问，它可以实现类之间的子类型关系；protected 继承使得基类的 public 成员不能被派生类的对象访问，但可以被派生类的派生类访问；private 继承使得基类的 public 成员既不能被派生类的对象访问，也不能被派生类的派生类访问。protected 和 private 继承主要用于实现上的继承，即纯粹为了代码复用。

3. 在 C++ 中，虚函数的作用是什么？

答：略

4. 在多继承中，什么情况下会出现二义性？怎样消除二义性？

答：在多继承中会出现两个问题：名冲突和重复继承。在多继承中，当多个基类中包含同名的成员时，它们在派生类中就会出现名冲突问题；在多继承中，如果直接基类有公共的基类，就会出现重复继承，这样，公共基类中的数据成员在多继承的派生类中就有多个拷贝。在 C++ 中，解决名冲突的方法是用基类名受限；解决重复继承问题的手段是采用虚基类。

5. 什么是虚基类？它有什么作用？

答：略

6. 如何定义两个类 A 和 B（B 是 A 的派生类），使得在程序中能够创建一个与指针变量 p（类型为 A *）所指向的对象是同类的对象？

答：

```
class A
{ ...
public:
    virtual A *create()
    { return new A;
    }
};

class B:public A
{ ...
public:
    A *create()
    { return new B;
    }
};

int main()
{ A *p;

    if (...)
        p = new A;
    else
        p = new B;
    ...
}
```

```

A *q;
q = p->create(); //创建一个与p所指向的对象同类的对象。
...
}

```

二、填空题

1. 在 C++中，三种派生方式的说明符号为 public、protected、private 不加说明，则默认的派生方式为 private。
2. 当公有派生时，基类的公有成员成为派生类的 公有成员；保护成员成为派生类的 保护成员；私有成员成为派生类的 不可直接使用的成员。
当保护派生时，基类的公有成员成为派生类的 保护成员；保护成员成为派生类的 保护成员；私有成员成为派生类的 不可直接使用的成员。
3. 包含成员对象的派生类的初始化分为三个步骤：首先 调用基类的构造函数，其次 调用成员对象类的构造函数，最后 执行自己的构造函数。

三、选择题

1. 下面对派生类的描述中，错误的是（ **D** ）。
 - A. 一个派生类可以作为另外一个派生类的基类
 - B. 派生类至少有一个基类
 - C. 派生类的成员除了它自己的成员外，还包含了它的基类的成员
 - D. 派生类中继承的基类成员的访问权限到派生类中保持不变
2. 下列对友元关系叙述正确的是（ **AD** ）。——多选
 - A. 不能继承
 - B. 是类与类的关系
 - C. 是一个类的成员函数与另一个类的关系
 - D. 提高程序的运行效率
3. 当保护继承时，基类的（ **B** ）在派生类中成为保护成员，不能通过派生类的对象来直接访问。
 - A. 任何成员
 - B. 公有成员和保护成员
 - C. 公有成员和私有成员
 - D. 私有成员
4. 有如下类定义：


```

class MyBASE{
    int k;
public:
    void set(int n) { k=n; }
    int get( ) const { return k; }
};

class MyDERIVED: protected MyBASE{
protected:
    int j;
public:
    void set(int m, int n){ MyBASE::set(m); j=n; }
    int get( ) const{ return MyBASE::get( )+j; }
};

```

 则类 MyDERIVED 中保护成员个数是（ **B** ）。
 - A. 4
 - B. 3
 - C. 2
 - D. 1

5. 程序如下：

```
#include<iostream>
using namespace std;
class A {
    public:
        A() { cout<<"A"; }
};
```

```
class B {
    public:
        B() { cout<<"B"; }
};
```

```
class C: public A{
    B b;
    public:
        C() { cout<<"C"; }
};
```

```
int main() { C obj; return 0; }
```

执行后的输出结果是 (**D**)。

- A. CBA B. BAC C. ACB D. ABC

6. 类 O 定义了私有函数 F1。P 和 Q 为 O 的派生类，定义为 class P: protected O{...}; class Q: public O{...}。那么谁可以访问 F1: (**C**)

- A. O 的对象 B. P 类内 C. O 类内 D. Q 类内

7. 有如下类定义：

```
class XA{
    int x;
    public:
        XA(int n) { x=n; }
};
```

```
class XB: public XA{
    int y;
    public:
        XB(int a, int b);
};
```

在构造函数 XB 的下列定义中，正确的是 (**B**)。

- A. XB::XB (int a, int b): x(a), y(b){ }
- B. XB::XB (int a, int b): XA(a), y(b){ }
- C. XB::XB (int a, int b): x(a), XB(b){ }
- D. XB::XB (int a, int b): XA(a), XB(b){ }

8. 有如下程序：

```
#include<iostream>
using namespace std;
class Base{
```

```
private:
    void fun1( ) const { cout<<"fun1"; }
protected:
    void fun2( ) const { cout<<"fun2"; }
public:
    void fun3( ) const { cout<<"fun3"; }
};
```

```
class Derived : protected Base{
public:
    void fun4( ) const { cout<<"fun4"; }
};
```

```
int main(){
    Derived obj;
    obj.fun1( ); //①
    obj.fun2( ); //②
    obj.fun3( ); //③
    obj.fun4( ); //④
}
```

其中没有语法错误的语句是（ **D** ）。

- A. ① B. ② C. ③ D. ④

四、阅读题

1. 写出以下程序的运行结果

```
#include<iostream>
using namespace std;
class B1 {
public:
    B1(int i) { cout << "constructing B1" << i << endl; }
    ~B1( ) { cout << "destructing B1" << endl; }
};

class B2 {
public:
    B2( ) { cout << "constructing B3" << endl; }
    ~B2( ) { cout << "destructing B3" << endl; }
};

class C: public B2, virtual public B1 {
    int j;
public:
    C(int a, int b, int c): B1(a), memberB1(b), j(c) { }
private:
    B1 memberB1;
    B2 memberB2;
};
```

```
int main( ) {
    C obj(1,2,3);
}
```

答：该程序的运行结果为：

```
constructing B11
constructing B3
constructing B12
constructing B3
destructing B3
destructing B1
destructing B3
destructing B1
```

2. 写出以下程序的运行结果

```
#include<iostream>

class A {
    public:
        int n;
};

class B: public A {};
class C: public A {};

class D: public B, public C {
    int getn() { return B::n; }
};

void main()
{   D d;
    d.B::n = 10;
    d.C::n = 20;
    cout << d.B::n << ", " << d.C::n << endl;
}
```

答：该程序的运行结果为：

10,20

3. 写出以下程序的运行结果

```
#include <iostream>

class A {
    int a;
public:
    A(int i) { a=i; cout << "constructing class A" << endl; }
    void print() { cout << a << endl; }
    ~A() { cout << "destructing class A" << endl; }
};
```

```

class B1: public A {
    int b1;
public:
    B1(int i, int j): A(i) {    b1=j; cout << "constructing class B1" << endl;    }
    void print()
    {
        A::print();
        cout << b1 << endl;
    }
    ~B1() {    cout << "destructing class B1" << endl;    }
};

```

```

class B2: public A {
    int b2;
public:
    B2(int i, int j): A(i) {    b2=j; cout << "constructing class B2" << endl;    }
    void print()
    {
        A::print();
        cout << b2 << endl;
    }
    ~B2() {    cout << "destructing class B2" << endl;    }
};

```

```

class C: public B1, public B2 {
    int c;
public:
    C(int i, int j, int k, int l, int m) : B1(i, j), B2(k, l), c(m)
    {
        cout << "constructing class C" << endl;
    }
    void print()
    {
        B1::print();
        B2::print();
        cout << c << endl;
    }
    ~C() {    cout << "destructing class C" << endl;    }
};

```

```

void main()
{
    C c1(1,2,3,4,5);
    c1.print();
}

```

答：该程序的运行结果为：

```

constructing class A
constructing class B1

```

```
constructing class A
constructing class B2
constructing class C
1
2
3
4
5
destructing class C
destructing class B2
destructing class A
destructing class B1
destructing class A
```

4. 写出下面程序的运行结果:

```
#include <iostream>
using namespace std;
class A
{   int m;
    public:
        A() { cout << "in A's default constructor\n"; }
        A(const A&) { cout << "in A's copy constructor\n"; }
        ~A() { cout << "in A's destructor\n"; }
};

class B
{   int x,y;
    public:
        B() { cout << "in B's default constructor\n"; }
        B(const B&) { cout << "in B's copy constructor\n"; }
        ~B() { cout << "in B's destructor\n"; }
};

class C: public B
{   int z;
    A a;
    public:
        C() { cout << "in C's default constructor\n"; }
        C(const C&) { cout << "in C's copy constructor\n"; }
        ~C() { cout << "in C's destructor\n"; }
};

void func1(C x)
{   cout << "In func1\n";
}

void func2(C &x)
{   cout << "In func2\n";
}
```

```

int main()
{   cout << "-----Section 1-----\n";
    C c;
    cout << "-----Section 2-----\n";
    func1(c);
    cout << "-----Section 3-----\n";
    func2(c);
    cout << "-----Section 4-----\n";
    return 0;
}

```

答: -----Section 1-----

in B's default constructor

in A's default constructor

in C's default constructor

-----Section 2-----

in B's default constructor

in A's default constructor

in C's copy constructor

In func1

in C's destructor

in A's destructor

in B's destructor

-----Section 3-----

In func2

-----Section 4-----

in C's destructor

in A's destructor

in B's destructor

5. 写出下面程序的运行结果:

```

#include <iostream>
using namespace std;
class A
{   int x,y;
    public:
        A() { cout << "in A's default constructor\n"; f(); }
        A(const A&) { cout << "in A's copy constructor\n"; f(); }
        ~A() { cout << "in A's destructor\n"; }
        virtual void f() { cout << "in A's f\n"; }
        void g() { cout << "in A's g\n"; }
        void h() { f(); g(); }
};

```

class B: public A

```

{   int z;
    public:

```



```

        B() { cout << "in B's default constructor\n"; }
        B(const B&) { cout << "in B's copy constructor\n"; }
        ~B() { cout << "in B's destructor\n"; }
        void f() { cout << "in B's f\n"; }
        void g() { cout << "in B's g\n"; }
};

```

```

void func1(A x)
{
    x.f();
    x.g();
    x.h();
}

void func2(A &x)
{
    x.f();
    x.g();
    x.h();
}

```

```

int main()
{
    cout << "-----Section 1-----\n";
    A a;
    A *p=new B;
    cout << "-----Section 2-----\n";
    func1(a);
    cout << "-----Section 3-----\n";
    func1(*p);
    cout << "-----Section 4-----\n";
    func2(a);
    cout << "-----Section 5-----\n";
    func2(*p);
    cout << "-----Section 6-----\n";
    delete p;
    cout << "-----Section 7-----\n";
    return 0;
}

```

答: -----Section 1-----

in A's default constructor

in A's f

in A's default constructor

in A's f

in B's default constructor

-----Section 2-----

in A's copy constructor

in A's f

in A's f

in A's g

in A's f

in A's g

in A's destructor

-----Section 3-----

in A's copy constructor

in A's f

in A's f

in A's g

in A's f

in A's g

in A's destructor

-----Section 4-----

in A's f

in A's g

in A's f

in A's g

-----Section 5-----

in B's f

in A's g

in B's f

in A's g

-----Section 6-----

in A's destructor

-----Section 7-----

in A's destructor